



Praxisarbeit

Programmiertechnik B

Philemon Ammann



Inhaltsverzeichnis

1	Management Summary	3
1.1	Projektziel und Umfang	3
1.2	Kernfunktionalitäten	3
1.3	Testphase.....	3
1.4	Projektergebnisse.....	3
1.5	Lessons Learned	3
1.6	Fazit und Ausblick	3
2	Anforderungen und Aufgabenstellung	4
2.1	Anforderungen.....	4
2.1.1	Funktionale Anforderungen.....	4
2.1.2	Nicht-funktionale Anforderungen.....	4
2.1.3	Projektumfang	5
2.2	Aufgabenstellung	5
3	Design	6
3.1	Entwurfsprozess.....	6
3.2	Struktur.....	9
3.2.1	Struktur des Labyrinths	9
3.2.2	Bewegungsstruktur	9
4	Implementierung	11
4.1	Projektstruktur.....	11
4.2	Implementierung der Kernfunktionen	11
4.2.1	Labyrinth-Initialisierung und Platzierung	11
4.2.2	Spielerbewegung	12
4.3	Fehlerbehandlung und Robustheit	13
4.4	Erweiterbarkeit	13
5	Test	14
5.1	Testmethoden.....	14
5.1.1	Systemtest	14
5.1.2	Testperson	14
5.2	Ergebnis	14
6	Lessons learned	15
6.1	Was lief gut	15
6.2	Was lief nicht gut.....	15
6.3	Was ändern ich nächstes Mal	15
6.4	Was behalten wir bei oder führen neu ein.....	16
6.5	Risiken	16
6.6	Fazit.....	16
7	Anhang	17

1 Management Summary

1.1 Projektziel und Umfang

Im Rahmen dieses Projekts wurde ein textbasiertes Labyrinth-Spiel in der Programmiersprache C entwickelt. Ziel war es, die in den Fächern Programmiertechnik A und B erlernten Kenntnisse praktisch anzuwenden und ein interaktives, konsolenbasiertes Spiel zu erstellen.

Der Projektumfang umfasste die vollständige Entwicklung des Spiels – von der Konzeption über die Implementierung bis hin zur Dokumentation – mit einem Arbeitsaufwand von rund 25 bis 30 Lernstunden.

1.2 Kernfunktionalitäten

Das entwickelte Spiel bietet folgende Hauptfunktionen:

1. Generierung eines zufälligen Labyrinths
2. Platzierung von Spieler, Schatz und Hindernissen
3. Interaktive Steuerung des Spielers über die Tastatur
4. Kollisionserkennung sowie Überprüfung der Siegbedingung

1.3 Testphase

Umfangreiche Tests während und nach der Entwicklung waren massgebend für den Projekterfolg. Das Spiel wurde zusätzlich von Testpersonen ausprobiert. Alle entdeckten Fehler wurden systematisch behoben, was zu einem stabilen und benutzerfreundlichen Endprodukt führte.

1.4 Projektergebnisse

Das Projekt wurde erfolgreich abgeschlossen, und alle definierten Anforderungen konnten erfüllt werden. Das resultierende Spiel ist funktional, benutzerfreundlich und deckt die gesetzten Lernziele ab. Die modulare Struktur des Codes ermöglicht zudem einfache zukünftige Erweiterungen.

1.5 Lessons Learned

Wichtige Erkenntnisse aus dem Projekt umfassen:

- Die Wichtigkeit einer klaren Struktur von Anfang an
- Die Bedeutung eines modularen Designs für Wartbarkeit und Erweiterbarkeit
- Den hohen Stellenwert umfassender Fehlerbehandlung und Tests
- Den Wert gut verständlicher Kommentierungen im Code
- Den Nutzen regelmässiger Versionskontrolle (GitHub, Codespaces)

1.6 Fazit und Ausblick

Das Projekt war ein voller Erfolg. Es bot wertvolle praktische Erfahrungen in der Programmierung und vertiefte das Verständnis für zentrale Konzepte und Strukturen.

Zukünftige Erweiterungsmöglichkeiten umfassen die Implementierung verschiedener Labyrinth-Layouts sowie zusätzlicher Spielelemente.

Zusammenfassend lässt sich sagen, dass das Projekt nicht nur die gesetzten Ziele erreicht hat, sondern auch eine solide Grundlage für zukünftige Entwicklungen in diesem Bereich geschaffen wurde.

2 Anforderungen und Aufgabenstellung

2.1 Anforderungen

In dieser Praxisarbeit wird ein C-Programm entwickelt, das ein Labyrinth als 2D-Raster darstellt und eine steuerbare Spielfigur enthält, die einen Schatz finden und Hindernissen ausweichen muss. Die folgenden Anforderungen werden in funktionale und nicht-funktionale Anforderungen unterteilt. Funktionale Anforderungen beschreiben, was das System tun soll, während nicht-funktionale Anforderungen festlegen, wie gut bzw. unter welchen Bedingungen es dies tun soll.

2.1.1 Funktionale Anforderungen

Die Erstellung des Labyrinths bildet das Fundament des Spiels. Dazu wird ein zweidimensionales Array verwendet, um das Labyrinth darzustellen. Innerhalb dieses Arrays werden verschiedene Elemente platziert:

- der Spieler, dargestellt durch den Buchstaben „P“ (für *Player*),
- der Schatz, dargestellt durch „T“ (für *Treasure*),
- Hindernisse, dargestellt durch „O“.

Die Position von Spieler und Schatz wird zu Beginn jedes Spiels zufällig bestimmt, um Spielinteresse und Abwechslung zu gewährleisten.

Die Bewegung des Spielers ist der zentrale Bestandteil des Spiels. Der Spieler kann mithilfe der Tasten W (oben), A (links), S (unten) und D (rechts) gesteuert werden. Zur Bestätigung der Eingabe muss zusätzlich die Return-Taste gedrückt werden. Dadurch werden die Spielzüge klar und kontrolliert ausgeführt. Nach jeder Bewegung wird das Labyrinth mit den aktuellen Positionen des Spielers neu angezeigt.

Ein wesentlicher Bestandteil ist die Kollisionserkennung. Das Programm muss sicherstellen, dass sich der Spieler weder durch Hindernisse bewegen noch das Labyrinth verlassen kann. Versucht der Spieler, ein solches Feld zu betreten, wird die Bewegung verhindert und eine entsprechende Meldung ausgegeben. Das Spielziel ist das Erreichen des Schatzes. Sobald der Spieler das Feld betritt, auf dem sich der Schatz befindet, endet das Spiel mit einer Siegesmeldung.

Das Programm gibt dem Spieler fortlaufend Eingabeaufforderungen, die über die möglichen Aktionen informieren.

2.1.2 Nicht-funktionale Anforderungen

Das Spiel wird in der Konsole ausgeführt und verzichtet bewusst auf eine grafische Benutzeroberfläche.

Ein wesentlicher Aspekt betrifft den Quellcode. Dieser soll klar strukturiert und ausführlich kommentiert sein, um Lesbarkeit und Wartbarkeit sicherzustellen. Die Verwendung modularer Funktionen trägt zur Übersichtlichkeit bei und erleichtert mögliche zukünftige Erweiterungen.

Ein weiteres Ziel ist die Benutzerfreundlichkeit. Das Spiel soll intuitiv bedienbar sein, sodass auch Personen ohne Vorerfahrung schnell einsteigen können. Klare und verständliche Anweisungen unterstützen dabei. Darüber hinaus ist die Robustheit des Programms von Bedeutung. Es soll in der Lage sein, Fehleingaben des Benutzers abzufangen und angemessen darauf zu reagieren, ohne dass es zu Abstürzen oder unerwartetem Verhalten kommt.

Schliesslich soll der Code so gestaltet sein, dass zukünftige Erweiterungen – beispielsweise unterschiedliche Labyrinth-Layouts oder zusätzliche Spielelemente – mit geringem Aufwand implementiert werden können.

2.1.3 Projektumfang

Der Projektumfang ist auf einen Arbeitsaufwand von rund 25 bis 30 Lernstunden ausgelegt. In diesem Zeitraum soll ein vollständig funktionsfähiges Konsolenspiel entstehen, das sämtliche zuvor definierten Anforderungen erfüllt.

Neben der Programmierung umfasst der Projektumfang auch die Erstellung einer Dokumentation, welche den gesamten Entwicklungsprozess beschreibt – von der Designphase über die Implementierung bis hin zur abschliessenden Testphase.

2.2 Aufgabenstellung

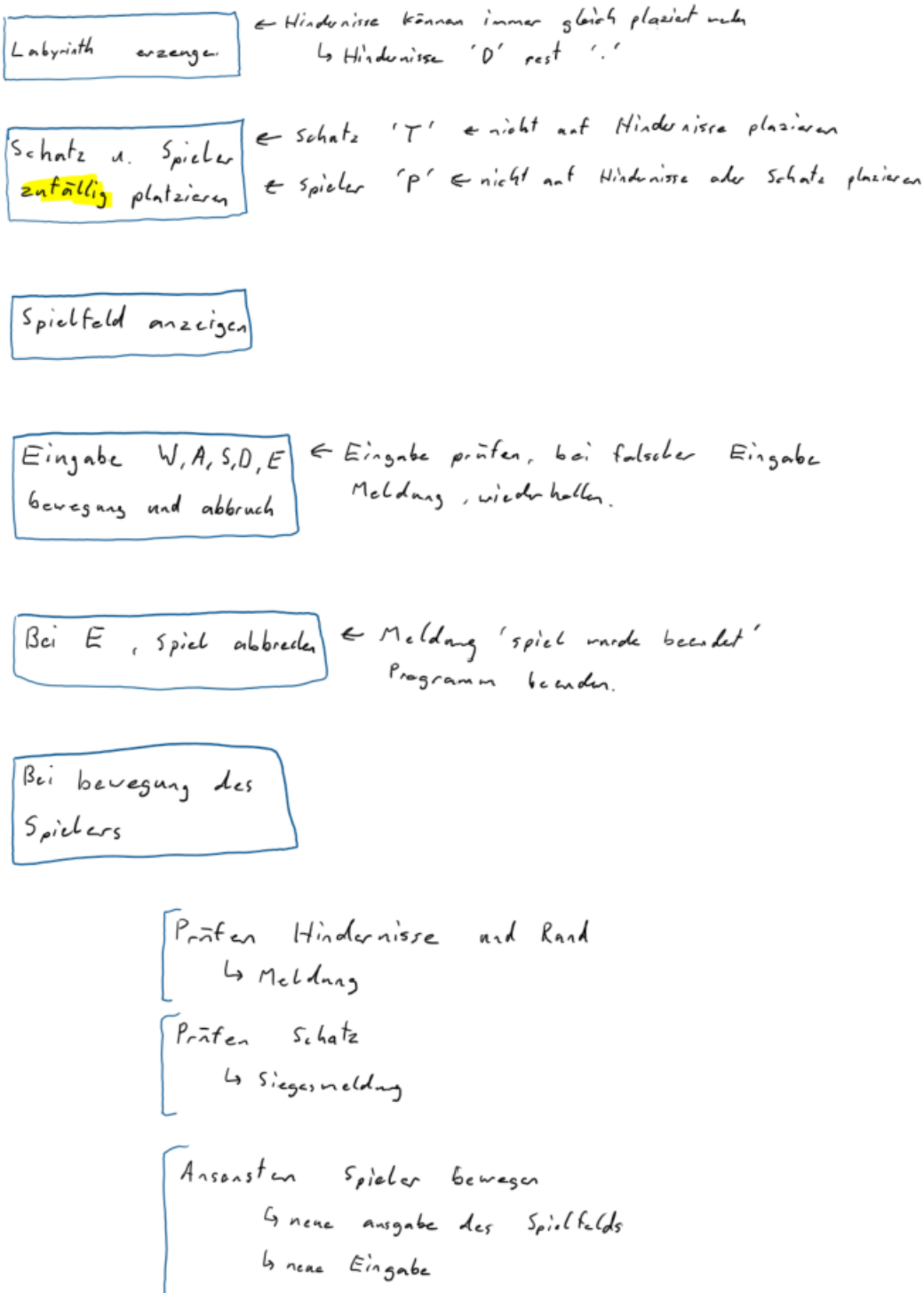
Im Rahmen dieser Praxisarbeit soll ein textbasiertes Labyrinth-Spiel entwickelt werden, das als Konsolenapplikation in der Programmiersprache C implementiert wird. Das Hauptziel dieses Projekts besteht darin, die in den Fächern Programmiertechnik A und B erlernten Kenntnisse praktisch anzuwenden und gleichzeitig ein interaktives und authentisches Spielerlebnis zu schaffen.

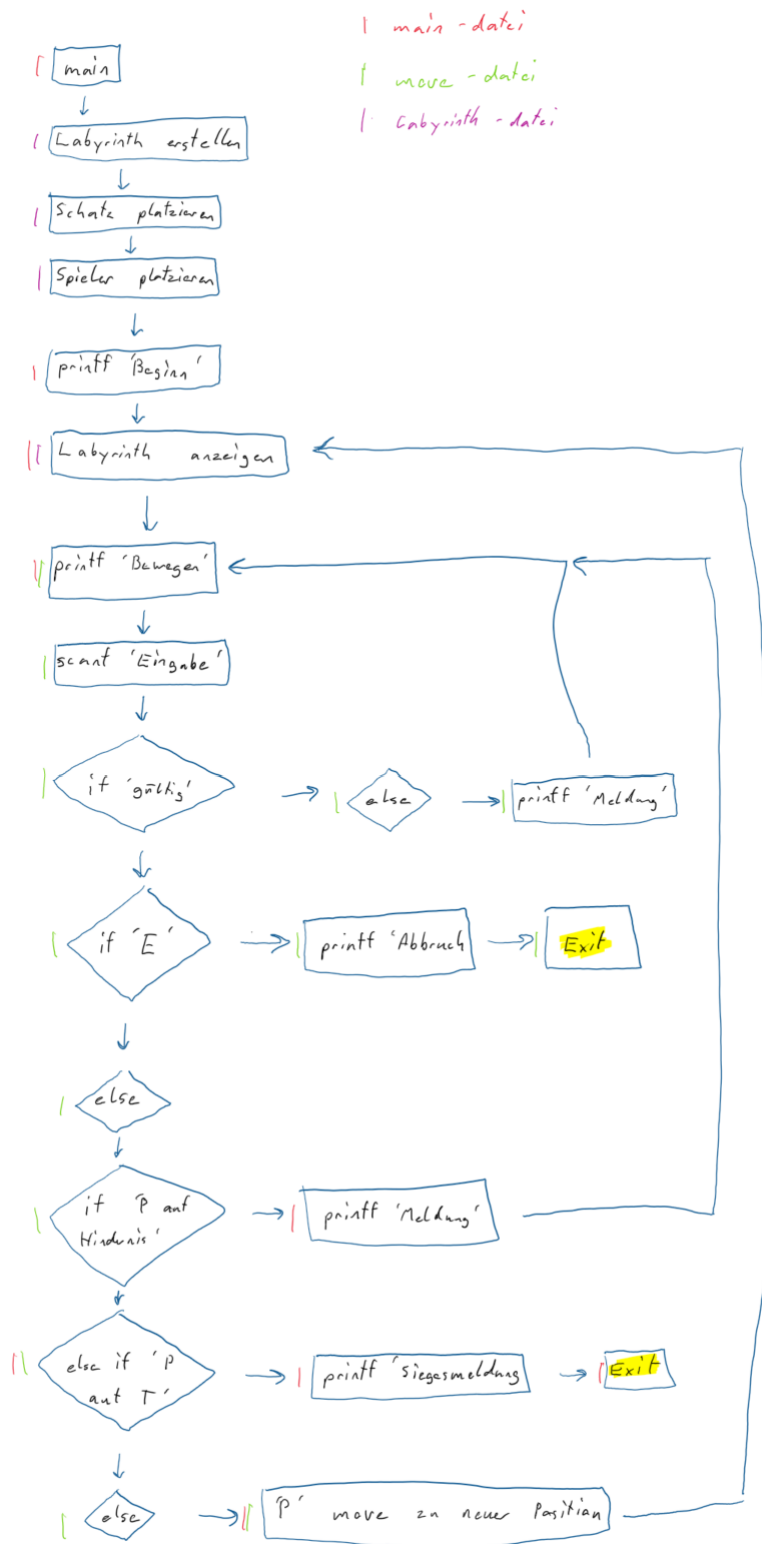
Das zu entwickelnde Spiel versetzt den Spieler in ein virtuelles Labyrinth, das durch ein zweidimensionales Array repräsentiert wird. In diesem Labyrinth soll sich der Spieler bewegen können, mit dem Ziel, einen versteckten Schatz zu finden. Die Herausforderung besteht darin, den Weg durch das Labyrinth zu navigieren, während der Spieler Hindernissen ausweicht.

3 Design

3.1 Entwurfsprozess

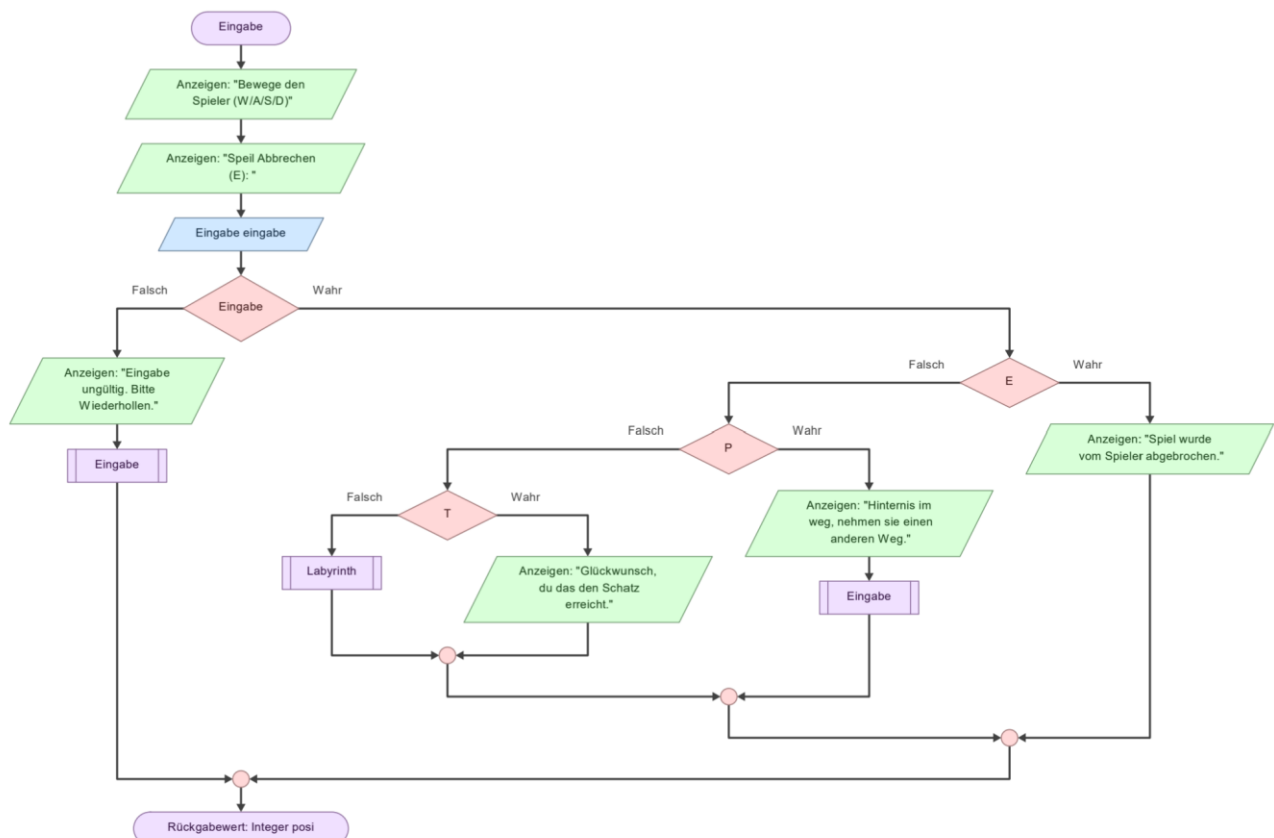
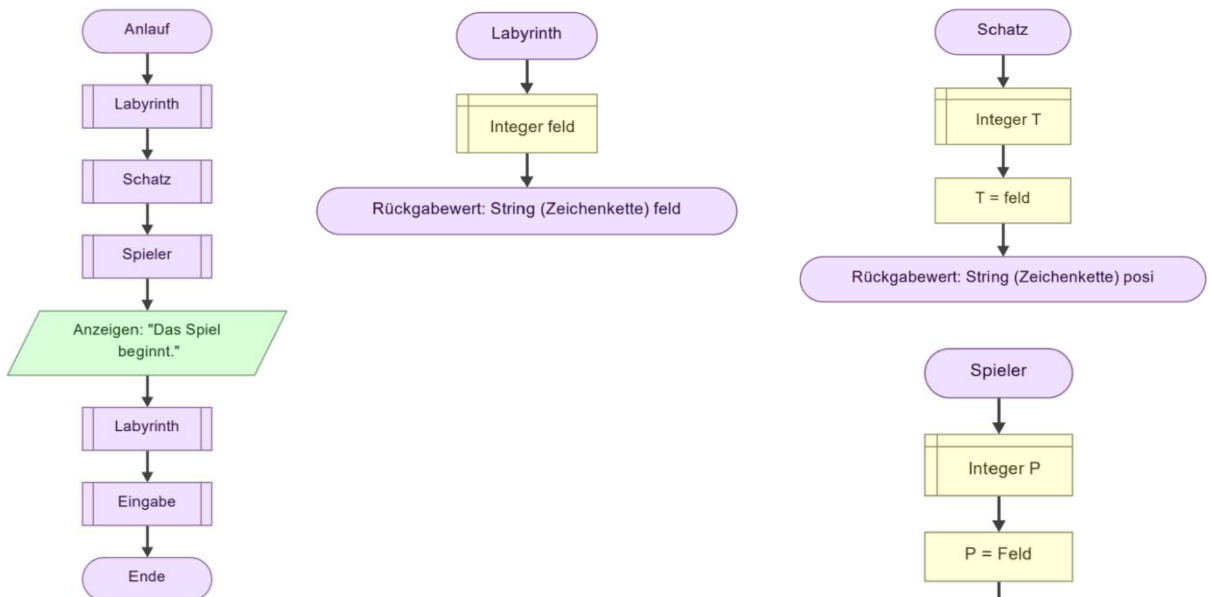
Der Entwurfsprozess für das Labyrinth-Spiel begann mit dem Zusammentragen der Anforderungen und der Aufgabenstellung. Basierend auf diesen Informationen wurden erste Skizzen und Ideen eines möglichen Ablaufes des Programms erstellt und anschliessend in einem Diagramm dargestellt.





Die drei farbigen Striche waren meine erste Überlegung bei der Skizze, wie ich das Programm genau in die verschiedenen Quellcodedateien aufteilen werde. Wenn man im Nachhinein einen Vergleich zwischen Code und Skizze macht, merkt man, dass die Skizze nur grob ist und nicht alles genau so umgesetzt werden konnte oder wurde. Sie war lediglich ein Anhaltspunkt und Rahmen, hat sich aber letztlich als mein Hauptleitfaden erwiesen.

Beim Diagramm, das mit Flowgorithm erstellt wurde, ist anzumerken, dass man dort nur in Funktionen arbeiten kann. Eine Aufteilung auf verschiedene Quellcodedateien ist daher nicht möglich. Deshalb zeigt das Diagramm hauptsächlich den Ablauf der Eingaben und der Kollisionskontrolle. Alles ist darin vereinfacht dargestellt und nicht in allen Details präzise. Da ich hauptsächlich mit der Skizze gearbeitet habe, wurde das Flowgorithm-Diagramm nie vollständig nachgeführt, weshalb weiterhin Fehler enthalten sind. Es dient in erster Linie der Veranschaulichung, wie ein Ablaufdiagramm mit einem solchen Programm aussehen könnte.



3.2 Struktur

Man erkennt in der Skizze gut den groben Ablauf, dem das Programm während des Spiels folgt:

1. Erstellen eines Labyrinths.
Dabei erfolgt auch die Platzierung der Hindernisse sowie des Schatzes und des Spielers.
2. Aufforderung an den Spieler, eine Bewegung einzugeben.
3. Suchen der Spielerposition im Feld; anschliessend Verarbeiten der Eingabe des Benutzers auf ihre Gültigkeit (Schatz gefunden oder falsche Eingabe).
4. Bei korrekter Eingabe, ohne Abbruchkriterium, wird der Spieler neu platziert.
5. Ausgabe des Labyrinths auf der Konsole.
6. Rückkehr zu Schritt 2 mit einer erneuten Aufforderung an den Spieler, eine Bewegung einzugeben.

Das Ziel war es, die Struktur möglichst einfach und klar zu halten.

3.2.1 Struktur des Labyrinths

Das Labyrinth wird in der Quelldatei labyrinth.c erstellt. Dabei werden die Hindernisse zunächst manuell eingefügt und anschliessend der Schatz sowie der Spieler zufällig platziert. Das Labyrinth ist als zweidimensionales Array aufgebaut.

Es wird darauf geachtet, dass der Schatz nicht auf einem Hindernis platziert wird. Ebenso wird beim Einfügen des Spielers sichergestellt, dass er weder auf einem Hindernis noch auf dem Schatz positioniert wird.

Hier sieht man das Endergebnis:

```
Das Spiel beginnt.
Labyrinth:
. . . . . T O . . . .
O . . . . . . . . .
O . . O . . . . . .
. . . . . O O . . . .
O . O . . . . . . .
O O . O O O . . O O
. . . O . . O . . . .
O . . . . O . O O .
. . . . . O . . . .
. . O . O P . . . O
Bewege den Spieler (A</D>/W^/S) - Spiel Abbrechen (E):
```

3.2.2 Bewegungsstruktur

Nach einer Aufforderung zur Eingabe durch den Spieler wird zunächst die Position des Spielers ermittelt. Anschliessend wird die Eingabe geprüft: Zunächst, ob der Spieler eine unzulässige Bewegung ausführen möchte – zum Beispiel das Verlassen des Feldes oder das Betreten eines Hindernisses – oder ob er den Schatz gefunden hat.

Der Ablauf gestaltet sich wie folgt:

1. Ortung des Spielers
2. Prüfung der Eingabe
3. Wenn die Eingabe korrekt ist: Prüfung auf Exit
4. Wenn nicht Exit: Prüfung auf Hindernis oder Spielfeldrand
5. Wenn kein Hindernis und innerhalb des Feldes: Prüfung auf Schatz
6. Wenn kein Schatz: Ausführung der Bewegung per Switch-Case. Danach erneute Ausgabe des Labyrinths und eine neue Aufforderung zur Eingabe.

Der Spieler wird im Labyrinth durch seine Position „P“ dargestellt:

```
Labyrinth:
. . . . . 0 . . .
0 . . . . . . . .
0 . . 0 . . . . .
. . . . 0 0 . . . .
0 . 0 . . . . . . .
0 0 . 0 0 0 . . 0 0
. . 0 . . 0 . . P
0 . . . . 0 . 0 0 .
. . . . . 0 . . .
. . 0 . 0 . T . . 0
Bewege den Spieler ( A < / D > / W ^ / S ) - Spiel Abbrechen (E): d
Labyrinth:
. . . . . 0 . . .
0 . . . . . . . .
0 . . 0 . . . . .
. . . . 0 0 . . . .
0 . 0 . . . . . . .
0 0 . 0 0 0 . . 0 0
. . 0 . . 0 . . P
0 . . . . 0 . 0 0 .
. . . . . 0 . . .
. . 0 . 0 . T . . 0
Bewege den Spieler ( A < / D > / W ^ / S ) - Spiel Abbrechen (E): s
Labyrinth:
. . . . . 0 . . .
0 . . . . . . . .
0 . . 0 . . . . .
. . . . 0 0 . . . .
0 . 0 . . . . . . .
0 0 . 0 0 0 . . 0 0
. . 0 . . 0 . . P
0 . . . . 0 . 0 0 .
. . . . . 0 . . .
. . 0 . 0 . T . . 0
```

Oben ist die normale Bewegung des Spielers dargestellt. Im unteren Bild werden die Meldungen angezeigt, die bei einem Hindernis oder beim Verlassen des Spielfeldes ausgegeben werden.

```
Labyrinth:
. . . . . 0 . . .
0 . . . . . . . .
0 . . 0 . . . . .
. . . . 0 0 . . . .
0 . 0 . . . . . . .
0 0 . 0 0 0 . . 0 0
. . 0 . . 0 . . P
0 . . . . 0 . 0 0 .
. . . . . 0 . . .
. . 0 . 0 . T . . 0
Bewege den Spieler ( A < / D > / W ^ / S ) - Spiel Abbrechen (E): d
Du kannst nicht aus dem Feld rauslaufen, bitte eine andere Richtung wählen.
Bewege den Spieler ( A < / D > / W ^ / S ) - Spiel Abbrechen (E): a
Hindernis im weg, nehmen sie einen anderen Weg.
Bewege den Spieler ( A < / D > / W ^ / S ) - Spiel Abbrechen (E): w
Labyrinth:
. . . . . 0 . . .
0 . . . . . . . .
0 . . 0 . . . . .
. . . . 0 0 . . . .
0 . 0 . . . . . . .
0 0 . 0 0 0 . . 0 0
. . 0 . . 0 . . P
0 . . . . 0 . 0 0 .
. . . . . 0 . . .
. . 0 . 0 . T . . 0
```

4 Implementierung

Dieser Abschnitt bietet einen erweiterten Einblick in die Implementierung des Labyrinth-Spiels und zeigt die wichtigsten Codebeispiele.

Die Implementierung des Labyrinth-Spiels erfolgte auf Basis des im vorherigen Abschnitt beschriebenen Designs. Der Code wurde in mehrere Module aufgeteilt, um eine klare Struktur und eine einfache Wartbarkeit zu gewährleisten. Im Folgenden werden die wichtigsten Aspekte der Implementierung erläutert. Für detailliertere Gedankengänge und den vollständigen Ablauf sollten der Quellcode und die Kommentare herangezogen werden.

4.1 Projektstruktur

Das Projekt besteht aus folgenden Dateien:

- main.c: Enthält die main()-Funktion und startet den Spielablauf.
- labyrinth.h und labyrinth.c: Definieren und implementieren die Struktur sowie die Funktionen für das Labyrinth.
- move.h und move.c: Behandeln die Spielerlogik und -bewegung.

Es wurden absichtlich wenige Module erstellt, damit das Projekt überschaubar bleibt: Das Erstellen und Darstellen des Labyrinths erfolgt im Modul labyrinth, die Bewegungs- und Kontrolllogik im Modul move. Bei umfangreicheren Codes würde man das Ganze selbstverständlich weiter aufteilen.

4.2 Implementierung der Kernfunktionen

4.2.1 Labyrinth-Initialisierung und Platzierung

Die Funktion labyBuild() in labyrinth.c erstellt das Labyrinth, initialisiert alle Felder und platziert die Hindernisse sowie Schatz („T“) und Spieler („P“):

```

1 void labyBuild() {
2     for(int i=0; i<10; i++) { // initialisiere Feld
3         for(int j=0; j<10; j++) {
4             feld[i][j] = '.'; // . als Platzhalter
5         }
6     }
7
8     // Hindernisse platzieren, ganzes feld[10][10], random manuell
9     feld[0][6] = '0';
10
11     feld[1][0] = '0';
12
13     feld[2][0] = '0';
14     feld[2][3] = '0';
15
16     feld[3][4] = '0';
17     feld[3][5] = '0';
18
19     feld[4][0] = '0';
20     feld[4][2] = '0';
21
22     feld[5][0] = '0';
23     feld[5][1] = '0';
24     feld[5][3] = '0';
25     feld[5][4] = '0';
26     feld[5][5] = '0';
27     feld[5][8] = '0';
28     feld[5][9] = '0';
29
30     feld[6][3] = '0';
31     feld[6][6] = '0';
32
33     feld[7][0] = '0';
34     feld[7][5] = '0';
35     feld[7][7] = '0';
36     feld[7][8] = '0';
37
38     feld[8][6] = '0';
39
40     feld[9][2] = '0';
41     feld[9][4] = '0';
42     feld[9][9] = '0';
43
44     // Schatz random platzieren, nicht auf Hindernis
45     int schatzX = rand() % 10; // Zufallszahl 0-9
46     int schatzY = rand() % 10;
47
48     while(feld[schatzX][schatzY] == '0') { // solange Hindernis, neu generieren
49         schatzX = rand() % 10;
50         schatzY = rand() % 10;
51     }
52     feld[schatzX][schatzY] = 'T'; // T = (Schatz)
53
54     // Spieler random platzieren, nicht auf Hindernis oder Schatz
55     int spielerX = rand() % 10;
56     int spielerY = rand() % 10;
57     while(feld[spielerX][spielerY] == '0' || feld[spielerX][spielerY] == 'T') { // solan
58         spielerX = rand() % 10;
59         spielerY = rand() % 10;
60     }
61     feld[spielerX][spielerY] = 'P'; // P = Spieler
62 }

```

Dabei wird das zweidimensionale Array erstellt, die Hindernisse werden gesetzt und anschliessend werden Schatz und Spieler zufällig platziert. Natürlich hätte man das Array auch auf eine andere Art manuell füllen können, was wiederum Codezeilen eingespart hätte.

Ich habe es jedoch bewusst so belassen, wie es zu Beginn umgesetzt wurde. Damit möchte ich zeigen, dass man auf manche Ideen erst später kommt und dass Vereinfachungen jederzeit möglich sind.

4.2.2 Spielerbewegung

Die Funktion `eingabe()` in `move.c` verarbeitet die Bewegungseingaben des Spielers:

```

6 // Spielerposition Variablen initialisieren/deklarieren
7 int spielerX = 0; // Startposition X
8 int spielerY = 0; // Startposition Y
9
10 // interne Spielerposition ermitteln
11 void find_player(int *x, int *y) {
12     for (int i = 0; i < 10; ++i) {
13         for (int j = 0; j < 10; ++j) {
14             if (feld[i][j] == 'P') {
15                 *x = i; *y = j; // Spielerposition gefunden
16                 return;
17             }
18         }
19     }
20     *x = 0; *y = 0; // falls Spieler nicht gefunden wird, setze auf (0,0)
21 }
22
23 int eingabe(void) {
24     find_player(&spielerX, &spielerY); // aktuelle Spielerposition ermitteln
25     char c;
26     printf("Bewege den Spieler (A/D/W/S) - Spiel Abbrechen (E): ");
27     scanf("%c", &c); // nur ein Zeichen einlesen
28     // Eingabepuffer leeren
29     if (c != '\n') { //der rest der hinter dem eichen ist, wird gelöscht
30         while (getchar() != '\n');
31     }
32
33     /*das meiste laeuft eigentlich hier ab, halt von der eingabe zur bewegung, zum labyrinth aus
34     mein ziel wahr es eigentlich mal moeglichst wenig rekursionen zu machen, aber hier ging es halt
35     aber das gefuehl mir nicht so. daher bin ich dann wieder zurueck auf mein ursprung geagnen.*/
36
37     if (eingabe) {
38         if (c == 'E' || c == 'e') {
39             printf("Spiel wurde vom Spieler abgebrochen.\n\n");
40             _Exit(0);
41         }
42         else if (c == 'W' || c == 'A' || c == 'S' || c == 'D' || c == 'w' || c == 'a' || c == 's' || c == 'd') {
43             if (c == 'W' || c == 'w') {
44                 move('W');
45                 labyrinth(feld);
46                 eingabe();
47             }
48             else if (c == 'A' || c == 'a') {
49                 move('A');
50                 labyrinth(feld);
51                 eingabe();
52             }
53             else if (c == 'S' || c == 's') {
54                 move('S');
55                 labyrinth(feld);
56                 eingabe();
57             }
58             else if (c == 'D' || c == 'd') {
59                 move('D');
60                 labyrinth(feld);
61                 eingabe();
62             }
63             else {
64                 printf("Eingabe ungültig. Bitte Wiederholen.\n\n");
65                 eingabe();
66             }
67             return 0;
68         }
69         else {
70             printf("Eingabe ungültig. Bitte Wiederholen.\n\n");
71             eingabe();
72         }
73     }
74     return 0;
75 }
76
77 void move(char richtung){
78
79     int nx = spielerX;
80     int ny = spielerY;
81
82     /*ja ich weiss, hier koennte man noch 2-3 funktionen generieren, die alles schmaeler machen,
83     ich habe es extra so gelassen, wie ich es zum ersten mal programmiert habe.*/
84     switch (richtung) {
85
86         case 'W':
87             // move up
88             richtung == nx--;
89             if (nx < 0 || nx >= 10) { // Spielrand prüfen
90                 printf("Du kannst nicht aus dem Feld rauslaufen, bitte eine andere Richtung wählen\n");
91                 eingabe();
92             }
93             if (feld[nx][ny] == 'O') { // Hindernis prüfen
94                 printf("Hindernis im weg, nehmen sie einen anderen Weg.\n\n");
95                 eingabe();
96             }
97             if (feld[nx][ny] == 'T') { // Schatz gefunden prüfen
98                 printf("Glückwunsch, du das den Schatz erreicht.\n\n");
99                 _Exit(0);
100             }
101             feld[spielerX][spielerY] = '.'; // alten Spielerpunkt löschen
102             spielerX--; // Spielerposition ändern
103             feld[spielerX][spielerY] = 'P'; // neuen Spielerpunkt setzen
104             break;

```

Sie bildet den Hauptteil des Programms. Zunächst wird die Eingabe eingelesen und auf Gültigkeit geprüft. Anschliessend wird kontrolliert, ob sich der Spieler auf ein Hindernis zubewegt, den Spielfeldrand überschreitet oder den Schatz erreicht hat. Zudem ist diese Funktion dafür verantwortlich, dass das Spiel so lange weiterläuft, bis eine andere Funktion aufgerufen wird, die das Spiel beendet.

4.3 Fehlerbehandlung und Robustheit

Um das Programm robust zu halten, wurden verschiedene Mechanismen implementiert:

- Platzieren des Schatzes erfolgt nicht auf einem Hindernis
- Platzieren des Spielers erfolgt nicht auf einem Hindernis oder auf dem Schatz
- Neuplatzierung des Spielers, falls dieser nicht gefunden wird
- Überprüfung, ob die Eingabe einem vorgegebenen Zeichen entspricht
- Abfangen, wenn der Spielrand erreicht wird
- Abfangen, wenn ein Hindernis im Weg steht
- Abfangen von Fehlern im switch-case-Block, auch wenn diese bereits durch die Eingabeprüfung verhindert werden sollten
- Abfangen möglicher Fehler in if-Anweisungen

4.4 Erweiterbarkeit

Der modulare Aufbau des Codes ermöglicht einfache Erweiterungen in der Zukunft. Diese würden keine zu grossen Veränderungen hervorrufen oder geben zumindest eine Struktur vor, die als Vorlage für einen komplexeren Code verwendet werden kann.

Mögliche Erweiterungen könnten sein:

- Implementierung verschiedener Labyrinth-Layouts
 - o Hinzufügen einer dritten Dimension
 - o Hindernisse, die sich bewegen
 - o Hindernisse, die bei jedem Spielstart zufällig gesetzt werden
 - o Ein Schatz, der sich bei jeder Bewegung ebenfalls verschiebt
 - o Monster, die den Spieler jagen
 - o und vieles mehr
- Spielerstatistik und Spielername mit Speicherung des Scores
- Erstellung des Spiels als lauffähiges Programm, das direkt per Mausklick gestartet werden kann

Diese Erweiterungen könnten durch Hinzufügen neuer Funktionen in die bestehenden Module oder durch das Erstellen neuer Module realisiert werden, ohne die Grundstruktur des Spiels wesentlich verändern zu müssen.

Im code+ wurde bereits begonnen, Erweiterungen zu integrieren und Verbesserungen vorzunehmen. Dies ist jedoch nicht Teil des eigentlichen Kernauftrags, sondern ein Zusatz – ohne Anspruch auf Vollständigkeit oder Fehlerfreiheit.

5 Test

Der Testprozess ist ein wichtiger Bestandteil der Entwicklung, um die Qualität und Zuverlässigkeit des Programms sicherzustellen und gegebenenfalls Fehler aufzudecken sowie zu korrigieren.

5.1 Testmethoden

5.1.1 Systemtest

Die Systemtests umfassten die Überprüfung des gesamten Spielablaufs von Start bis Ende. Dabei wurden verschiedene Szenarien durchgespielt, um sicherzustellen, dass das Spiel in allen möglichen Situationen korrekt funktioniert. Diese Tests wurden mehrfach vom Entwickler selbst durchgeführt.

5.1.2 Testperson

Es ist immer vorteilhaft, eine Entwicklung von einer aussenstehenden Person testen zu lassen. Der Entwickler handelt nie genau wie ein Endbenutzer, da er das gesamte Hintergrundwissen besitzt und dadurch kaum Bedienungsfehler verursacht.

Da das Programm von der Konsole aus gestartet werden muss, ist dafür bereits eine gewisse Erfahrung mit Programmierung erforderlich. Deshalb wurde der Programmstart durch den Entwickler durchgeführt.

Anschliessend konnte die Testperson selbständig eingreifen und das Spiel bis zum Ende spielen.

Dies verlief sehr gut und war für die Testperson einfach und logisch. Positiv war auch, dass die Eingabe zusätzlich mit Kleinbuchstaben möglich ist. Andernfalls wären dort Probleme aufgetreten, da die meisten Personen die grossen Buchstaben auf der Tastatur sehen und diese drücken – ohne zu bedenken, dass sie ohne aktiviertes Capslock automatisch Kleinbuchstaben eingeben.

5.2 Ergebnis

Das mehrfache Testen war äusserst hilfreich, denn erst dadurch konnten verschiedene kleinere Fehler und Unstimmigkeiten in der Darstellung aufgedeckt werden. So zeigte sich beispielsweise, dass bestimmte Eingaben nicht wie vorgesehen verarbeitet wurden oder einzelne Ausgaben in der Konsole unklar wirkten. Diese Probleme wurden behoben, wodurch die Klarheit des Programms deutlich verbessert werden konnte.

Im Rahmen der Tests wurden unterschiedliche Szenarien durchgespielt: Bewegungen gegen den Spielfeldrand, das Treffen auf Hindernisse, der sofortige Fund des Schatzes sowie längere Spielsitzungen mit vielen Zügen. In allen Fällen reagierte das Programm wie vorgesehen, ohne abzustürzen oder unvorhersehbares Verhalten zu zeigen. Auch absichtlich falsche Eingaben wurden korrekt abgefangen, sodass die Robustheit des Programms bestätigt werden konnte.

Am Ende ist ein stabiles, zuverlässiges und benutzerfreundliches Konsolenspiel entstanden. Es erfüllt die zuvor definierten Anforderungen vollständig, lässt sich intuitiv bedienen und konnte auch von Testpersonen problemlos gespielt werden. Damit ist sichergestellt, dass das Projekt nicht nur technisch funktioniert, sondern auch in der Praxis den gewünschten Nutzen bietet.

6 Lessons learned

Insgesamt war es ein sehr spannendes Projekt, bei dem ich erstmals etwas Eigenes und etwas Grösseres entwerfen konnte. Natürlich traten Hindernisse und Schwierigkeiten auf, doch im Allgemeinen verlief die Entwicklung erstaunlich flott. Schade war lediglich, dass solche Projekte oft genau dann anstehen, wenn man bereits viele andere Verpflichtungen parallel hat.

6.1 Was lief gut

Was bei mir besonders gut lief, war der Start. Ich habe zwar viel Zeit in die Einarbeitung und in die Auseinandersetzung mit der Struktur investiert, doch im Nachhinein erwies sich das als sehr hilfreich. Dadurch ging die eigentliche Umsetzung erstaunlich flott voran.

Das Programm funktionierte bereits ab den ersten Versuchen fast wie geplant. Natürlich nicht ganz fehlerfrei, aber durch die klare Struktur und die vielen Überlegungen zu möglichen Problemen, die ich schon im Vorfeld bedacht hatte, liessen sich Schwierigkeiten schnell erkennen und beheben.

6.2 Was lief nicht gut

Natürlich gab es auch kleinere Probleme, die ich mir selbst durch das Programmieren eingebaut oder im Vorfeld nicht ausreichend bedacht hatte. Ein Beispiel war das Setzen des Spielers: Er wurde zwar zunächst zufällig platziert, doch bei der ersten Bewegung startete er immer oben in der Ecke, da ich ihn so initialisiert hatte.

Auch der Einsatz des Flowgorithm-Programms war weniger erfolgreich als erhofft. Mein Ziel war es, möglichst viel darin abzubilden, um den generierten Quellcode anschliessend weiterverwenden zu können. Das funktionierte jedoch nicht wie gewünscht, weshalb ich den Ansatz aufgeben musste.

Das grösste Problem war jedoch die Zeit. Zu Beginn hatte ich noch ausreichend Kapazität, um die Aufgaben gründlich zu lösen, und das Programm entwickelte sich sehr gut. Für den Feinschliff am Ende fehlte mir jedoch die nötige Zeit, da parallel viele andere Verpflichtungen anstanden.

6.3 Was ändere ich nächstes Mal

Eigentlich würde ich beim nächsten Mal nicht allzu viel anders machen. Mit GitHub und Codespaces war ich sehr produktiv, konnte einfach arbeiten und kam schnell voran.

Das Wenige, was ich in Zukunft anpassen würde, betrifft vor allem die Arbeitsumgebung: ein sauber eingerichtetes Visual Studio Code sowie allenfalls ein grösserer Bildschirm, der mehr Übersicht bietet. Auch ein anderes Diagramm- oder Flowchart-Programm, das einfacher aufgebaut ist als Flowgorithm, wäre von Vorteil. Wenn man mit Flowgorithm nicht regelmässig arbeitet, verliert man zu viel Zeit damit, die Funktionen des Programms wieder neu zu lernen, anstatt es effizient anzuwenden.

Ein weiterer Punkt ist die Zeitplanung. Je nachdem, ob ein Projekt im Rahmen des Studiums oder im Arbeitsumfeld entsteht, unterscheidet sich auch der verfügbare Zeitrahmen. Damit meine ich vor allem, wann und unter welchen Bedingungen man daran arbeitet. Nach der regulären Arbeit nochmals im Büro konzentriert am Projekt zu sitzen, ist etwas ganz anderes, als wenn man die Aufgaben während der Arbeitszeit erledigen kann.

6.4 Was behalten wir bei oder führen neu ein

Was ich auf jeden Fall beibehalten werde, ist der Schritt, zunächst von Hand einen Ablauf der Struktur aufzuzeichnen – ganz egal, wie komplex das Projekt ist. Damit habe ich immer einen klaren Anhaltspunkt, an dem ich starten kann. Diese Vorgehensweise hat mir sehr geholfen und wird auch in Zukunft Teil meiner Arbeitsweise bleiben.

Darüber hinaus möchte ich künftig versuchen, den Code noch modularer aufzubauen. In diesem Projekt war die Struktur von vornherein recht klar, und deshalb waren auch die Funktionen logisch benannt und gut zuzuordnen. In Zukunft möchte ich die Funktionen jedoch allgemeiner halten, sodass bei komplexeren Projekten eine höhere Wiederverwendbarkeit möglich ist.

Ein weiterer Punkt betrifft den Umgang mit Rekursionen. Ich möchte vermehrt darauf achten, keine rekursiven Funktionen zu schreiben. Dies hängt auch mit meinem angestrebten Arbeitsbereich zusammen, in dem rekursive Ansätze nicht erwünscht sind.

6.5 Risiken

Ich hatte mehrfach das Problem, dass ich Codespaces verwendet habe, der darin enthaltene Code jedoch nicht mehr ganz aktuell war. Dadurch kam es beim Speichern beziehungsweise beim Pushen am Ende zu Fehlermeldungen wegen Inkonsistenzen. Diese konnte ich zwar beheben, doch die Erfahrung zeigt, dass man besser regelmässig pusht und zu Beginn jeweils einen Pull macht, um sicherzugehen, dass man mit der aktuellen Version arbeitet. Das ist sicherlich etwas, woran ich mich noch stärker üben und worauf ich künftig genauer achten muss.

Natürlich gab es auch Risiken direkt im Code selbst – beispielsweise beim Bewegen des Spielers. Solche Kleinigkeiten fallen oft erst später auf und müssen dann mühsam gesucht und behoben werden.

6.6 Fazit

Es war ein sehr gutes und für mich sehr lernreiches Projekt, das mir in vielerlei Hinsicht geholfen hat – auch bei Dingen, die ich zuvor eher belächelt habe, wie zum Beispiel das Skizzieren einer Struktur, bevor man überhaupt mit dem Programmieren beginnt. Durch diese Vorgehensweise habe ich erkannt, wie wertvoll eine saubere Vorbereitung für den gesamten Entwicklungsprozess ist.

Darüber hinaus konnte ich mich durch dieses Projekt stark in der Programmierung vertiefen. Besonders der modulare Aufbau, die Arbeit mit mehreren Quellcodedateien sowie die systematische Fehlersuche haben mir wichtige praktische Erfahrungen vermittelt. Auch die Auseinandersetzung mit Versionskontrolle über GitHub und Codespaces war lehrreich und hat mir gezeigt, wie entscheidend sauberes Arbeiten mit solchen Tools ist.

Die Spannung und Freude an der Mikrocontroller- und Embedded-Programmierung ist durch dieses Projekt keineswegs kleiner geworden – im Gegenteil: Sie wurde noch verstärkt. Das Projekt hat mir gezeigt, dass ich auf dem richtigen Weg bin und dass mir diese Art von Arbeit sowohl fachlich als auch persönlich Spass macht und mich motiviert, weiter in diesem Bereich zu lernen und zu arbeiten.

7 Anhang

- Quellcode (originaler Hauptcode, «code») mit Kommentaren
- Flowgorithm-Datei
- code+ (Erweiterung mit Zusatzfunktionen, nicht Bestandteil der Arbeit)

Die relevanten Informationen stammen aus der Ausgangslage (Aufgabenstellung im PDF), den Unterlagen aus dem Unterricht (Programmiertechnik A und B) sowie dem Buch C als erste Programmiersprache (ISBN 978-3-658-45208-7). Unterstützung bei der Ausformulierung und Korrektur der Grammatik erfolgte durch LanguageTool und ChatGPT.